

Func<TEntity, bool> vs Expression<Func<TEntity, bool>>

Summary — why FindBy() uses Expression instead of a plain Func

1. The problem Func alone can't solve

A plain **Func<TEntity, bool>** is a delegate — a variable that holds a reference to a compiled, runnable method. It's great for filtering data that's already sitting in memory (like a List<Student>), because C# can just run it directly. But EF Core doesn't work on data in memory — it needs to turn your condition into a real SQL query so the **database** does the filtering. A compiled Func can only be *executed*, not *read* — EF Core can't peek inside it to build SQL from it.

2. What Expression does differently

Wrapping the same lambda in **Expression<Func<TEntity, bool>>** tells the C# compiler: don't compile this into runnable code — instead, build a data structure (an *expression tree*) that describes what the lambda says, step by step.

```
Expression<Func<Student, bool>> condition = s => s.Age > 18;
```

Instead of compiled instructions, this becomes a tree describing: take parameter **s** → access its **Age** property → compare it with **18** using **>**. That tree is just data describing the logic, not the logic actually running yet.

3. Why that matters in practice

Because the expression tree is readable data, EF Core can walk through it and generate real SQL, e.g. SELECT * FROM Students WHERE Age > 18. The filtering happens inside the database — fast and efficient. If a plain Func had been used instead, EF Core would have no choice but to load **every** row into memory first, then run the compiled function on each one in C# — much slower, especially on large tables.

4. Side-by-side summary

	Func<TEntity, bool>	Expression<Func<TEntity, bool>>
What it is	Compiled, runnable method	A data structure describing the logic
Where it runs	In your app's memory (C#)	Translated into SQL, runs in the database
Use case	Filtering an in-memory List<>	Filtering an EF Core DbSet<> / table

Bottom line: In FindBy(Expression<Func<TEntity, bool>> predicate), wrapping the condition in Expression<...> is what lets EF Core turn a lambda like s => s.Age > 18 into a real SQL WHERE clause, instead of pulling the entire table into memory first.